

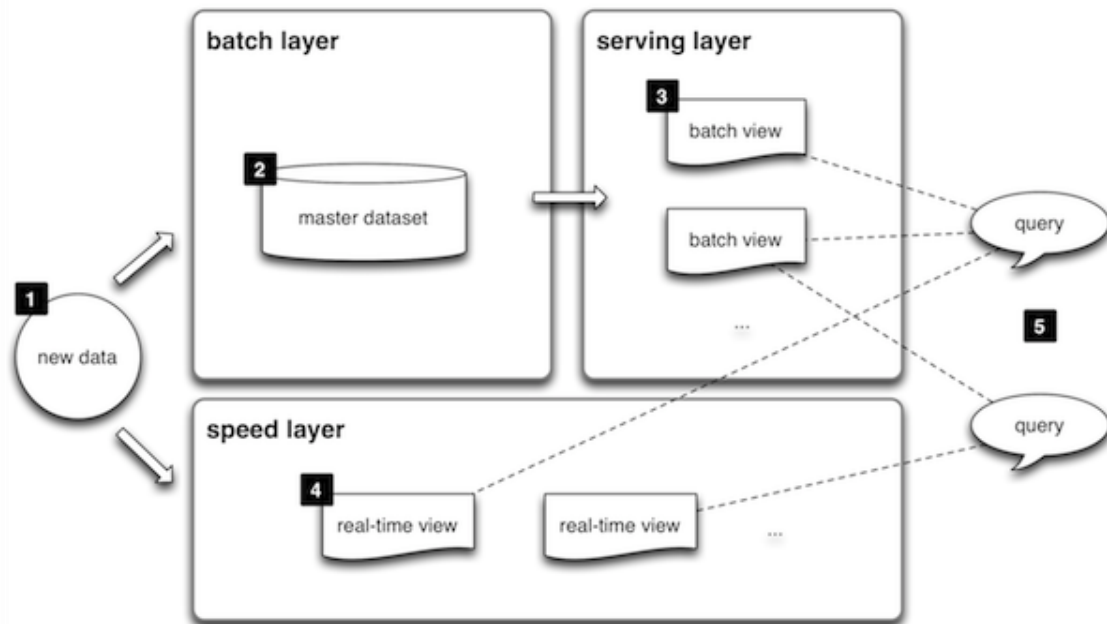
TD7: Ingeniería de Datos

Arquitectura Lambda.
Organización del Master Dataset.

Speed Layer

Serving Layer

Batch Layer



Organización del Master Dataset

Organización del Master Dataset



— — —

La **batch layer** está pensada para poder consumir el **master dataset** en su totalidad (y así generar batch views).

Pero eso no quiere decir que no haya necesidad de organizar los archivos con los datos de alguna forma, por dos razones:

- ❖ Muchos procesos van a generar **batch views** a partir de sólo una **porción** de *all data* y podemos optimizarlos.
- ❖ Los sistemas de cómputo distribuido padecen del problema de los **archivos pequeños** (**small-files problem**)

Particionado por verticales



— — —

Para contemplar el hecho de que distintos procesos consumen distintas partes del dataset, se pueden establecer divisiones de los datos a partir de sus características: el **particionado vertical**

- ❖ Por ejemplo, se podría querer procesar información recogida sólo en las últimas dos semanas.
- ❖ El almacenamiento batch debería permitir particionar la data de manera de procesar sólo aquella que es relevante para el caso.
- ❖ Contribuye a hacer los batch layer más eficientes.

Particionado por verticales



- ❖ Distintos tipos de eventos / hechos pueden almacenarse en carpetas separadas (plays, likes, logins, etc)
- ❖ Dentro de un mismo vertical, se puede particionar por tiempo (por mes, por día, por hora)

Particionado por verticales



— — —
No es un requerimiento de la batch layer,
pero:

- ❖ Optimiza el uso de recursos del filesystem (reduce la cantidad de transfers necesarios)
- ❖ Optimiza el uso de recursos de cómputo (procesar menos archivos implica menos procesos)
- ❖ Facilita el archivado de datos históricos

```
master-dataset
├── content-interactions
│   ├── 2024-05-23
│   ├── 2024-05-24
│   │   ├── 00001.parquet
│   ├── 2024-05-25
│   │   ├── 00001.parquet
│   │   ├── 00002.parquet
│   │   └── 00003.parquet
│   ├── 2024-05-26
│   └── 2024-05-27
│       ├── 00001.parquet
│       └── 00002.parquet
├── logins
│   ├── 2024-05-23
│   ├── 2024-05-24
│   │   ├── 00001.parquet
│   │   └── 00002.parquet
│   ├── 2024-05-25
│   ├── 2024-05-27
│   └── search-actions
│       ├── 2024-05-26
│       └── 2024-05-27
```

Particionado por verticales



Reduce transferencias innecesarias

Supongamos que se quiere analizar únicamente los inicios de sesión. Si todo estuviera mezclado:

```
master-dataset/  
    eventos.parquet
```

tendrías que leer un archivo enorme que contiene:

- ❖ logins
- ❖ búsquedas
- ❖ interacciones de contenido
- ❖ etc.

En cambio, con particionado por verticales:

```
master-dataset/logins/
```

va directamente a la carpeta relevante.

Particionado por verticales



Apache Parquet es un formato de almacenamiento de datos diseñado para analítica. Es algo parecido a un CSV, pero mucho más eficiente para Big Data.

nombre	edad	ciudad
Ana	25	Rosario
Juan	30	Córdoba
Pedro	40	Rosario

Un CSV guarda
todo fila por fila:

Ana,25,Rosario
Juan,30,Córdoba
Pedro,40,Rosario

Parquet, en cambio, almacena por columnas:

nombre: Ana, Juan, Pedro
edad: 25, 30, 40
ciudad: Rosario, Córdoba, Rosario

Como los valores similares quedan juntos, comprime muy bien. Por ejemplo:

ciudad
Rosario
Rosario
Rosario

se comprime muchísimo mejor que en CSV.

Áreas y consolidación



— — —

Un problema muy común en los sistemas de cómputo y almacenamiento distribuido es el **small-files problem**:

- ❖ Sistemas como HDFS se apoyan en el **block-size** para **distribuir el cómputo** entre nodos
- ❖ Los bloques de un archivo son todos (relativamente) iguales salvo el último (que puede ser mucho más chico)
- ❖ Cada archivo implica tantas **tareas** (por ejemplo de Map en Map/Reduce) como **bloques** tenga
- ❖ Procesar un mismo volumen de datos distribuido en muchos **archivos chicos** (menores al tamaño del bloque) necesita muchas más tareas que un solo archivo grande: cada tarea implica un **overhead**

Áreas y consolidación



— — —

Para evitar la dispersión en archivos y organizar las **particiones verticales**, se pueden disponer áreas dentro del master dataset:

- ❖ **Landing:** donde llegan los datos en archivos relativamente chicos
 - Para que el área de landing no esté compuesta por archivos de un solo registro cada uno, se pueden acumular en **colas** (tolerantes a fallos)
- ❖ **Main:** donde los archivos están organizados en **verticales** y **consolidados** (muchos archivos chicos se transforman en algunos archivos grandes)
 - El procesamiento para consolidar y organizar los datos se apoya en la **misma infraestructura** que el que va a generar las batch views (y por lo tanto es igualmente escalable)

El autor propone un framework de administración del master dataset que contempla éstas y otras cuestiones (como la serialización) llamado **Pail**

Cierre



Bibliografía

 **“Big data: Principles and best practices of scalable realtime data systems”, N. Marz, J. Warren, 1st edition, 2015.**
 Capítulos 2, 3, 4 y 5